

Exercise 10: Linear Programming (Sparse Quantile Regression)

Optimization in Machine Learning

Reproducibility

Produced with Python 3.10 and R 4.4.1. To reproduce, install the pinned package versions below.

Python

```
pip install numpy==2.0.2 matplotlib==3.10.8 scipy==1.15.3
```

R

```
install.packages("remotes")
remotes::install_version("ggplot2", "3.5.1")
remotes::install_version("linprog", "0.9.6")
```

Setup

We have $n = 50$ univariate observations with skewed (gamma) errors:

$$x_i \sim \text{Uniform}(0, 1), \quad y_i = 2x_i + \text{Gamma}(\text{shape} = 1).$$

The skewed noise motivates *quantile* regression instead of OLS — we want to fit the conditional quantile of $y \mid x$, not the conditional mean.

Multivariate sparse quantile regression is

$$\min_{\beta \in \mathbb{R}^p} \frac{1}{n} \sum_{i=1}^n \rho_{\tau}(y^{(i)} - \beta_0 - \beta^{\top} \mathbf{x}^{(i)}) \quad \text{s.t.} \quad \|\beta\|_1 \leq t$$

with $\tau \in (0, 1)$ and the **pinball loss**

$$\rho_{\tau}(s) = \begin{cases} \tau \cdot s & s > 0 \\ -(1 - \tau) \cdot s & s \leq 0 \end{cases}$$

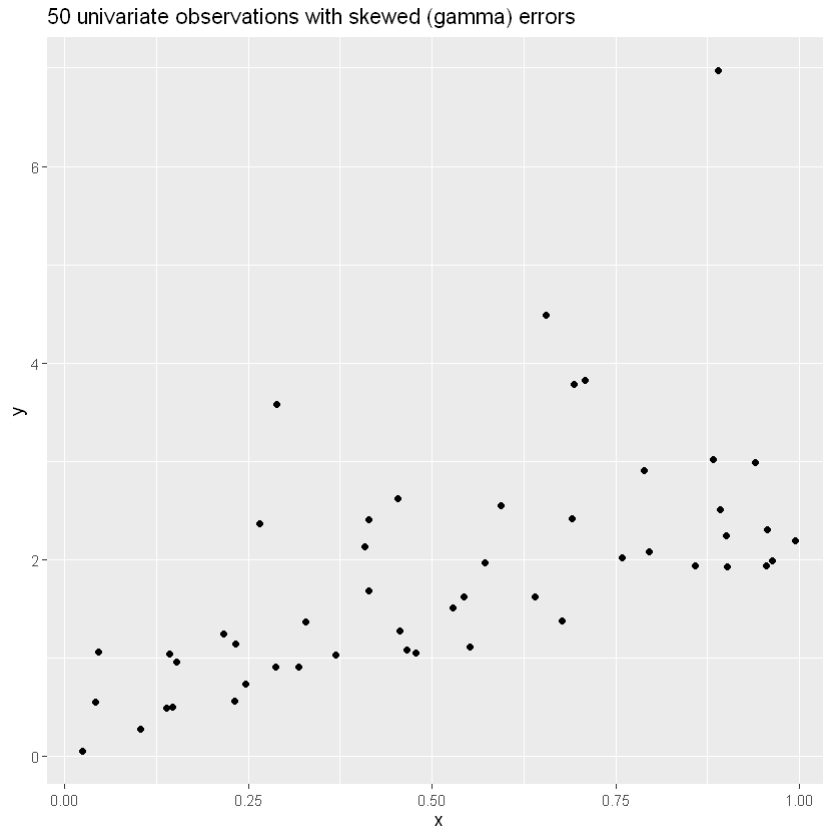
(asymmetric residual weighting; for $\tau = 0.5$ this is just $\frac{1}{2}|s|$). We treat the univariate ($p = 1$) case below.

R

```
library(ggplot2)

set.seed(123)
n <- 50
x <- runif(n)
y <- 2 * x + rgamma(n, shape = 1)

ggplot(data.frame(x = x, y = y), aes(x, y)) + geom_point() +
  ggtitle("50 univariate observations with skewed (gamma) errors")
```



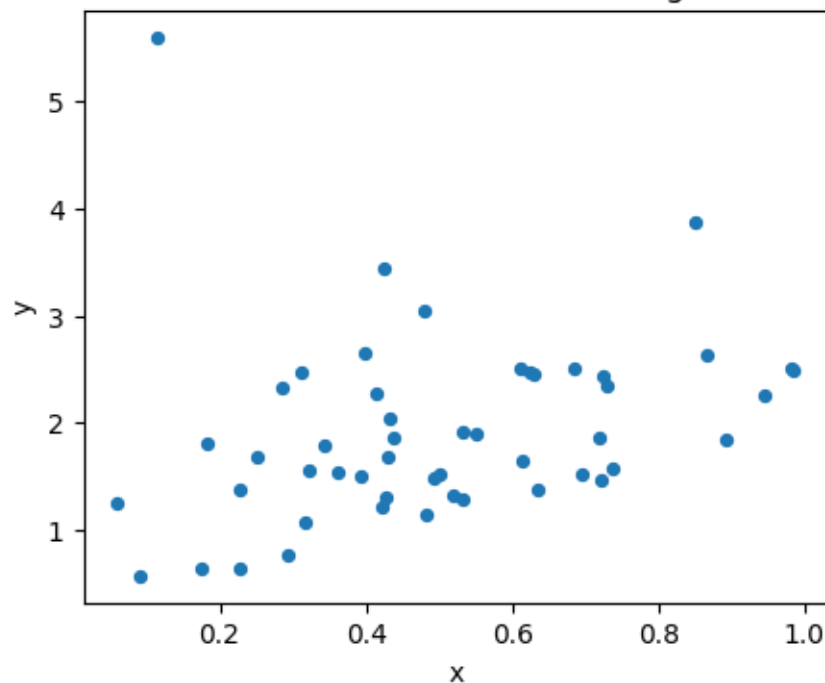
Python

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(123)
n = 50
x = np.random.uniform(size=n)
y = 2 * x + np.random.gamma(shape=1, size=n)

fig, ax = plt.subplots(figsize=(5, 4))
ax.scatter(x, y, s=18)
ax.set_xlabel("x"); ax.set_ylabel("y")
ax.set_title("50 univariate observations with skewed (gamma) errors")
plt.show()
```

50 univariate observations with skewed (gamma) errors



i Note

R vs. Python parity. R's `set.seed(123)` and Python's `np.random.seed(123)` use *different* RNGs, and `runif` / `rgamma` differ from `np.random.uniform` / `np.random.gamma` even given the same logical seed. The simulated data therefore differs between languages, so the LP solutions $\hat{\beta}$ will differ as well. Both implementations are correct; only the data differs. *Within* a language, the LP is deterministic — the same standard-form input gives identical $\hat{\beta}$ from `linprog` / `linprog`.

(a) Standard form of the LP

Find the standard LP form of the one-dimensional sparse quantile regression problem.

Hint: an unconstrained variable x can be decomposed into two non-negative variables $x = x^+ - x^-$, with $|x| = x^+ + x^-$. (At the optimum at most one of x^+, x^- is nonzero.)

Solution.

Univariate sparse quantile regression:

$$\min_{(\beta_0, \beta_1) \in \mathbb{R}^2} \frac{1}{n} \sum_{i=1}^n \rho_\tau(y^{(i)} - \beta_0 - \beta_1 x^{(i)}) \text{ s.t. } |\beta_1| \leq t$$

1. Decompose unconstrained parameters into positive and negative part: $\beta_i = \beta_i^+ - \beta_i^- \quad i = 0, 1$
2. Transform absolute value: $|\beta_1| \leq t \iff \beta_1^+ + \beta_1^- \leq t$
3. We can write $\rho_\tau(\underbrace{y^{(i)} - \beta_0 - \beta_1 x^{(i)}}_{=: r^{(i)}}) = \tau \cdot r^{(i)} \mathbb{1}_{\{r^{(i)} > 0\}} - (1 - \tau) \cdot r^{(i)} \mathbb{1}_{\{r^{(i)} \leq 0\}} = \tau \cdot r^{(i)+} + (1 - \tau) \cdot r^{(i)-}$
4. Transform equality of the residuals into two inequalities: $r^{(i)+} - r^{(i)-} = y^{(i)} - \beta_0 - \beta_1 x^{(i)} \iff r^{(i)+} - r^{(i)-} \leq y^{(i)} - \beta_0 - \beta_1 x^{(i)}$ and $-r^{(i)+} + r^{(i)-} \leq -y^{(i)} + \beta_0 + \beta_1 x^{(i)}$

With this we get the standard form:

$$\max_{\mathbf{z} \in \mathbb{R}^{4+2n}} \mathbf{c}^\top \mathbf{z}$$

s.t.

$$\mathbf{A}\mathbf{z} \leq \mathbf{b}, \quad \mathbf{z} \geq \mathbf{0}$$

with

$$\mathbf{z} = \begin{pmatrix} \beta_0^+ \\ \beta_0^- \\ \beta_1^+ \\ \beta_1^- \\ r^{(1)+} \\ \vdots \\ r^{(n)+} \\ r^{(1)-} \\ \vdots \\ r^{(n)-} \end{pmatrix}, \quad \mathbf{c} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ -\tau \\ \vdots \\ -\tau \\ -(1-\tau) \\ \vdots \\ -(1-\tau) \end{pmatrix}, \quad \mathbf{A} = \begin{pmatrix} 0 & 0 & 1 & 1 & 0 & \dots & 0 \\ 1_n & -1_n & \mathbf{x} & -\mathbf{x} & \mathbf{I}_n & & -\mathbf{I}_n \\ -1_n & 1_n & -\mathbf{x} & \mathbf{x} & -\mathbf{I}_n & & \mathbf{I}_n \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} t \\ \mathbf{y} \\ -\mathbf{y} \end{pmatrix}$$

(b) Visualise the empirical risk and feasible region

Plot $\mathcal{R}_{\text{emp}}(\beta_0, \beta_1)$ for $(\beta_0, \beta_1) \in [-3, 3]^2$ with $\tau = 0.4$. Mark the feasible region $|\beta_1| \leq t = 1.7$.

Solution.

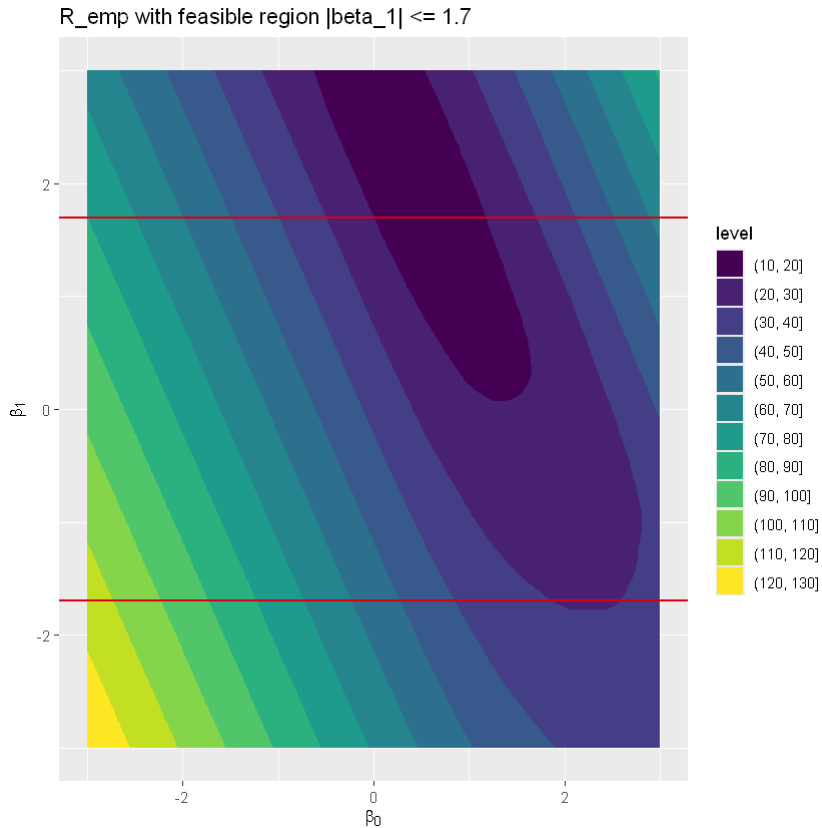
R

```
tau <- 0.4
tval <- 1.7

# Pinball loss
remp <- function(beta) {
  r <- y - beta[1] - beta[2] * x
  sum(ifelse(r > 0, tau * r, -(1 - tau) * r))
}

b_grid <- seq(-3, 3, by = 0.05)
bb <- expand.grid(X1 = b_grid, X2 = b_grid)
fbb <- apply(bb, 1, remp)
df_grid <- data.frame(bb, fbb = fbb)

remp_plot <- ggplot(df_grid, aes(X1, X2, z = fbb)) +
  geom_contour_filled() +
  geom_hline(yintercept = c(-tval, tval), color = "#D90012", linewidth = 0.7) +
  xlab(expression(beta[0])) + ylab(expression(beta[1])) +
  ggtitle("R_emp with feasible region  $|\beta_1| \leq 1.7$ ")
remp_plot
```



Python

```
tau = 0.4
tval = 1.7

def remp(beta):
    """Pinball loss summed over the data."""
    r = y - beta[0] - beta[1] * x
    return float(np.sum(np.where(r > 0, tau * r, -(1 - tau) * r)))

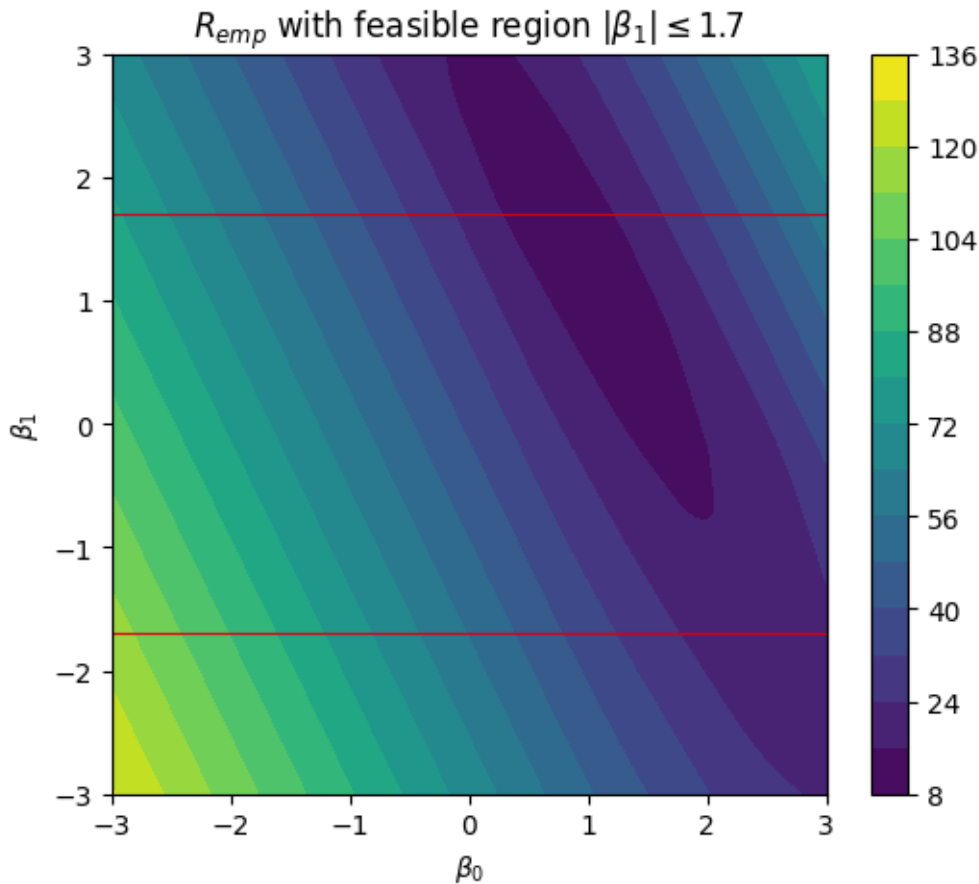
b_grid = np.arange(-3, 3 + 0.05, 0.05)
B0, B1 = np.meshgrid(b_grid, b_grid)
Z = np.array([[remp(np.array([b0, b1])) for b0 in b_grid] for b1 in b_grid])

fig, ax = plt.subplots(figsize=(6, 5))
```

```

cs = ax.contourf(B0, B1, Z, levels=14)
ax.axhline( tval, color="#D90012", linewidth=0.8)
ax.axhline(-tval, color="#D90012", linewidth=0.8)
ax.set_xlabel(r"$\beta_0$"); ax.set_ylabel(r"$\beta_1$")
ax.set_title(r"$R_{\text{emp}}$ with feasible region $|\beta_1| \leq 1.7$")
fig.colorbar(cs)
plt.show()

```



Two horizontal red lines at $\beta_1 = \pm t$ delimit the feasible region. The contour shows the level sets of $\mathcal{R}_{\text{emp}}$. The optimum lives on or inside the strip.

(c) Solve the LP

Use R's `linprog::solveLP` (or Python's `scipy.optimize.linprog`) to solve the sparse 40 % quantile regression with $t = 1.7$.

Solution.

R

```
library(linprog)

# Build the standard form (a):
#   z = (beta0+, beta0-, beta1+, beta1-, r1+, ..., rn+, r1-, ..., rn-)
#   c, A, b as in the prose above.
Amat <- c(0, 0, 1, 1, rep(0, 2 * n)) # budget
# r+ - r- <= y - beta_0 - beta_1 x
Amat <- rbind(Amat, cbind(1, -1, x, -x, diag(n), -diag(n)))
Amat <- rbind(Amat, cbind(-1, 1, -x, x, -diag(n), diag(n))) # mirrored
bvec <- c(tval, y, -y)
cvec <- c(0, 0, 0, 0, rep(tau, n), rep(1 - tau, n)) # positive c, minimize

res <- solveLP(cvec, bvec, Amat, maximum = FALSE, lpSolve = TRUE)

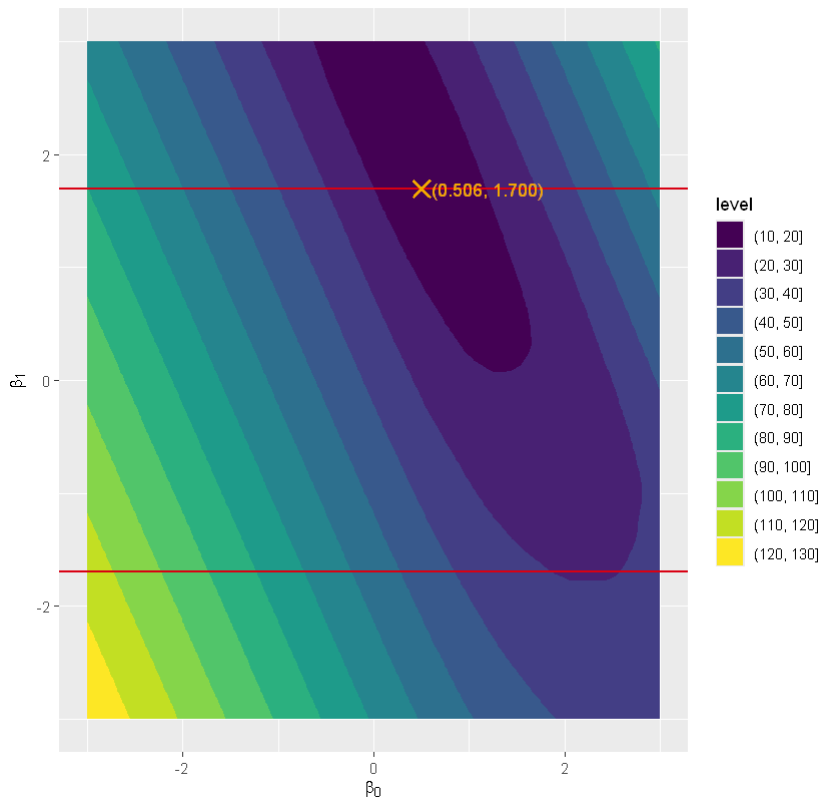
beta0 <- res$solution[1] - res$solution[2]
beta1 <- res$solution[3] - res$solution[4]
constraint_active <- abs(abs(beta1) - tval) < 1e-6

cat(sprintf("beta_hat = (%+.4f, %+.4f), R_emp* = %.4f\n",
            beta0, beta1, remp(c(beta0, beta1))))
cat(sprintf("budget constraint |beta_1| <= %.2f is %s (|beta_1*| = %.4f)\n",
            tval, if (constraint_active) "ACTIVE" else "slack", abs(beta1)))

# Contour from (b), now with the LP optimum marked in parameter space.
remp_plot +
  annotate("point", x = beta0, y = beta1, color = "#F2A800",
          size = 3.5, shape = 4, stroke = 1.5) +
  annotate("text", x = beta0, y = beta1,
          label = sprintf(" (%.3f, %.3f)", beta0, beta1),
          hjust = 0, color = "#F2A800") +
  ggtitle(sprintf("R_emp with feasible region; orange X = LP optimum (%s)",
                  if (constraint_active) "constraint active" else "interior"))
```

```
beta_hat = (+0.5062, +1.7000), R_emp* = 13.7880
budget constraint |beta_1| <= 1.70 is ACTIVE (|beta_1*| = 1.7000)
```

R_emp with feasible region; orange X = LP optimum (constraint active)



Python

```
from scipy.optimize import linprog

# Build the standard form (a):
#   z = (beta0+, beta0-, beta1+, beta1-, r1+, ..., rn+, r1-, ..., rn-)
#   (length 4 + 2n)
#   minimize c^T z subject to Az <= b, z >= 0.
I_n = np.eye(n)
row_budget = np.concatenate([[0, 0, 1, 1], np.zeros(2 * n)])
row_pos    = np.column_stack([np.ones(n), -np.ones(n), x, -x, I_n, -I_n])
row_neg    = np.column_stack([-np.ones(n), np.ones(n), -x, x, -I_n, I_n])
A = np.vstack([row_budget, row_pos, row_neg])
b = np.concatenate([[tval], y, -y])
c = np.concatenate([[0, 0, 0, 0], np.full(n, tau), np.full(n, 1 - tau)])

res = linprog(c, A_ub=A, b_ub=b, bounds=[(0, None)] * (4 + 2 * n),
```

```

        method="highs")

beta0 = res.x[0] - res.x[1]
beta1 = res.x[2] - res.x[3]
constraint_active = abs(abs(beta1) - tval) < 1e-6

print(f"beta_hat = ({beta0:+.4f}, {beta1:+.4f}), "
      f"R_emp* = {remp(np.array([beta0, beta1])):.4f}")
print(f"budget constraint |beta_1| <= {tval:.2f} is "
      f"{'ACTIVE' if constraint_active else 'slack'} "
      f"(|beta_1*| = {abs(beta1):.4f})")
print(f"linprog status: {res.message}, iters: {res.nit}")

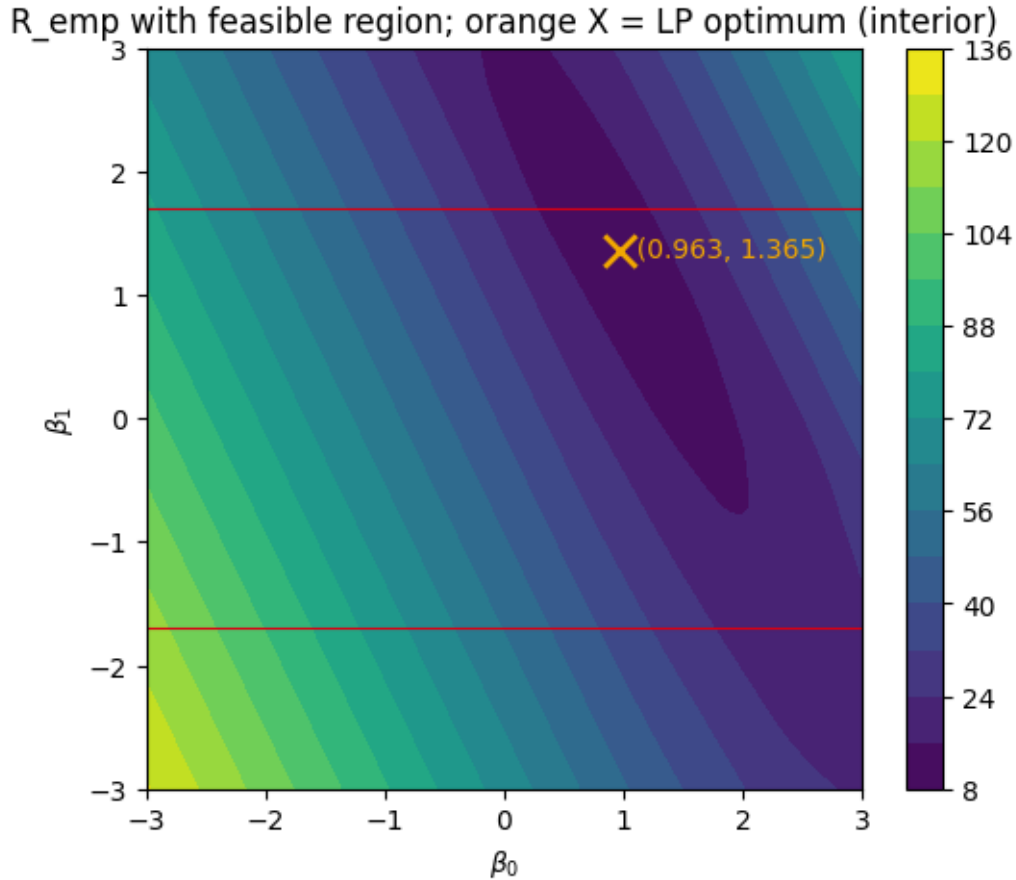
# Contour from (b), now with the LP optimum marked in parameter space.
fig, ax = plt.subplots(figsize=(6, 5))
cs = ax.contourf(B0, B1, Z, levels=14)
ax.axhline( tval, color="#D90012", linewidth=0.8)
ax.axhline(-tval, color="#D90012", linewidth=0.8)
ax.scatter(beta0, beta1, color="#F2A800", marker="x", s=140, linewidths=2)
ax.annotate(f" ({beta0:.3f}, {beta1:.3f})", (beta0, beta1), color="#F2A800",
           fontsize=10, va="center")
ax.set_xlabel(r"$\beta_0$"); ax.set_ylabel(r"$\beta_1$")
ax.set_title(f"R_emp with feasible region; orange X = LP optimum "
           f"({'constraint active' if constraint_active else 'interior'})")
fig.colorbar(cs)
plt.show()

```

```

beta_hat = (+0.9633, +1.3647), R_emp* = 12.6711
budget constraint |beta_1| <= 1.70 is slack (|beta_1*| = 1.3647)
linprog status: Optimization terminated successfully. (HiGHS Status 7: Optimal), iters: 50

```



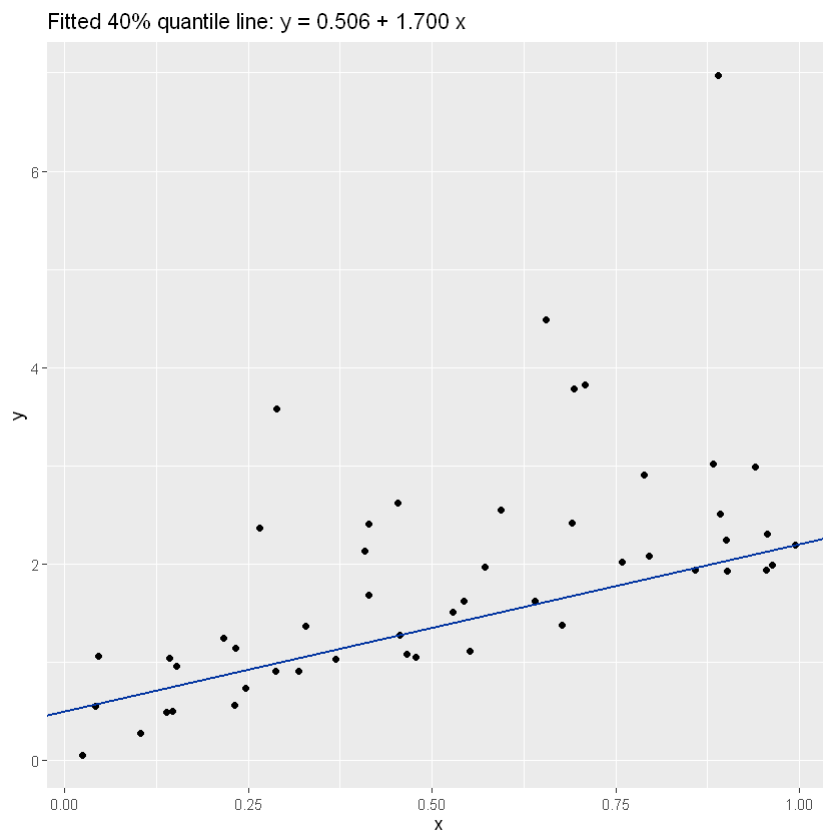
We construct $\mathbf{c}$, $\mathbf{A}$, $\mathbf{b}$ following (a), feed to the solver, and recover $\hat{\beta}_j = z_j^+ - z_j^-$. The fitted line $y = \hat{\beta}_0 + \hat{\beta}_1 x$ goes through the data; with $\tau = 0.4$ it lies *below* the $\tau = 0.5$ median fit (and well below the OLS fit, which the gamma-skewed errors would push high).

Whether the sparsity budget $|\beta_1| \leq t$ binds depends on the data. If the unconstrained quantile slope would exceed t , the optimum is clamped to the boundary $\beta_1 = \pm t$ (constraint *active*), so the plotted optimum lands exactly on the red line; otherwise the optimum lies strictly inside the feasible strip (constraint *slack*), below the line. Because R and Python draw different samples, the two panels here illustrate the two regimes — the printout states which one holds in each case.

R

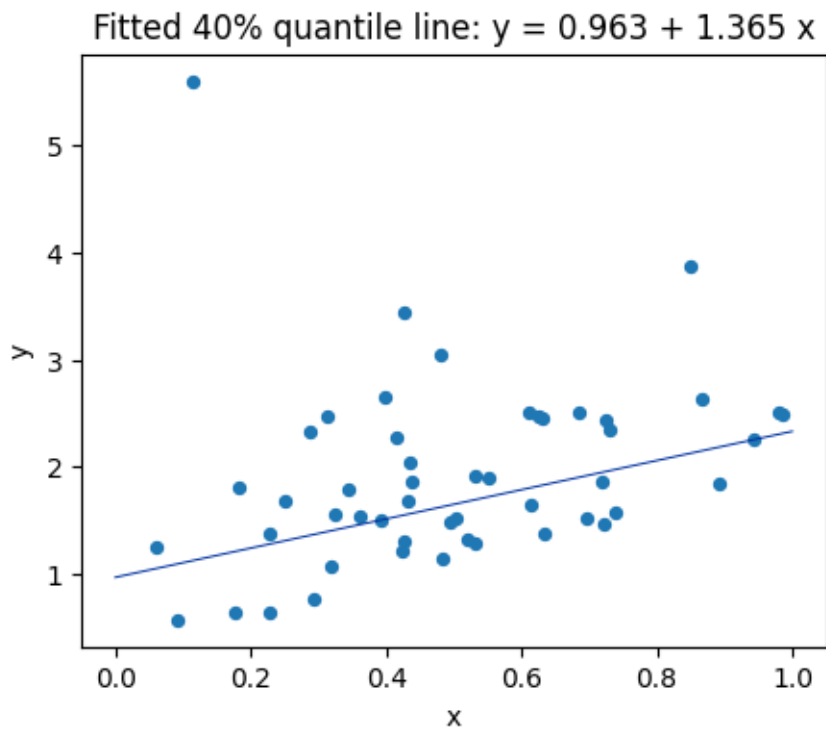
```
ggplot(data.frame(x = x, y = y), aes(x, y)) +
  geom_point() +
  geom_abline(intercept = beta0, slope = beta1,
```

```
color = "#0033A0", linewidth = 0.7) +
ggtitle(sprintf("Fitted 40%% quantile line: y = %.3f + %.3f x", beta0, beta1))
```



Python

```
fig, ax = plt.subplots(figsize=(5, 4))
ax.scatter(x, y, s=18)
x_range = np.array([0.0, 1.0])
ax.plot(x_range, beta0 + beta1 * x_range, color="#0033A0", linewidth=0.7)
ax.set_xlabel("x"); ax.set_ylabel("y")
ax.set_title(f"Fitted 40% quantile line: y = {beta0:.3f} + {beta1:.3f} x")
plt.show()
```



(d) Dual formulation

State the dual of the LP from (a).

Solution.

From the lecture, we know that the dual problem must be

$$\max_{\alpha \in \mathbb{R}^{2n+1}} -\alpha^\top \mathbf{b}$$

s.t.

$$\begin{aligned} -\alpha^\top \mathbf{A} &\leq -\mathbf{c}^\top \\ \alpha &\geq \mathbf{0} \end{aligned}$$